ENTROPY, INFORMATION AND OPTIMAL TRANSPORT PROBLEM SHEET

This sheet covers the lectures on entropy, information theory and optimal transport. Question 1 practises entropy and code lengths, Question 2 mutual information, the KL divergence and cross-entropy, and Question 3 the maximum entropy method. Each ends with a short Python part (`import numpy as np` is assumed). Question 4 is harder and optional: you write a small optimal transport solver in PyTorch and use it to move points by gradient descent. Logarithms are base 2 (bits) unless we say otherwise.

1

(a) Find the entropy of (i) a fair coin, (ii) a coin with $\mathbb{P}(\text{head}) = 0.9$, and (iii) a random variable X taking the values a, b, c, d with probabilities $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{8}$.

(i) $H = -2 \times \frac{1}{2}\log_2\frac{1}{2} = 1$ bit. (ii) $H = -0.9\log_2 0.9 - 0.1\log_2 0.1 \approx 0.137 + 0.332 = 0.469$ bits: the outcome is much less uncertain. (iii) $H = \frac{1}{2}\times 1 + \frac{1}{4}\times 2 + 2\times\frac{1}{8}\times 3 = 1.75$ bits.

[4 marks]

(b) We code the outcomes of X in part (a)(iii) as $a \to 0$, $b \to 10$, $c \to 110$, $d \to 111$. Why can a string of these code words always be decoded? Decode `0110100111`. Find the expected code length and compare it with the entropy.

It is a prefix code: no code word is the start of another, so reading from the left we always know where a code word ends. `0|110|10|0|111` is $acbad$. The expected length is $\frac{1}{2}\times 1 + \frac{1}{4}\times 2 + \frac{1}{8}\times 3 + \frac{1}{8}\times 3 = 1.75$ bits, exactly the entropy, because each code length is $-\log_2$ of its probability.

[4 marks]

(c) Instead we use two bits for every outcome (`00`, `01`, `10`, `11`). How many extra bits per outcome does this cost? What is the largest entropy any distribution on four outcomes can have, and which distribution has it?

The expected length is 2 bits, so 0.25 extra bits per outcome. (This is the KL divergence between X's distribution and the uniform distribution that the code is optimal for.) Entropy is concave and, by Jensen's inequality, at most $\log_2 4 = 2$ bits, reached only by the uniform distribution: for uniform outcomes the two-bit code is optimal.

[4 marks]

(d) X_1 and X_2 are independent copies of X. What is the entropy of the pair (X_1, X_2)?

For independent variables entropies add: $H = 1.75 + 1.75 = 3.5$ bits.

[3 marks]

(e) Write a Python function `entropy(p)` that returns the entropy in bits of a probability vector (treat $0\log 0$ as 0). Check part (a), then draw 10000 random distributions on four outcomes with `rng.dirichlet(np.ones(4))` and find the largest entropy.

```
def entropy(p):
    p = np.asarray(p, dtype=float)
    p = p[p > 0]    # 0 log 0 = 0
    return -(p * np.log2(p)).sum()
print(entropy([0.5, 0.5]), entropy([0.9, 0.1]))
print(entropy([0.5, 0.25, 0.125, 0.125]))
rng = np.random.default_rng(0)
H = [entropy(rng.dirichlet(np.ones(4)))
       for _ in range(10000)]
print(max(H))
```

This prints 1.0, 0.469 and 1.75, and the largest of the random entropies is 1.9998: close to, but never above, 2 bits.

[5 marks]

End of question 1

Answers

2

(a) Bits X and Y have the joint distribution

	$Y=0$	$Y=1$
$X=0$	0.4	0.1
$X=1$	0.1	0.4

Find H_X, H_Y, $H_{X,Y}$, $H_{X|Y}$ and the mutual information $I_{X;Y}$.

Both marginals are $(\frac{1}{2},\frac{1}{2})$, so $H_X = H_Y = 1$ bit.
$H_{X,Y} = -2 \times 0.4 \log_2 0.4 - 2 \times 0.1 \log_2 0.1 \approx 1.722$ bits. By the chain rule $H_{X|Y} = H_{X,Y} - H_Y \approx 0.722$ bits, and $I_{X;Y} = H_X - H_{X|Y} \approx 0.278$ bits.

[5 marks]

(b) Explain why this table describes sending equally likely bits down a binary symmetric channel. What is the error rate f, and what is the channel capacity $1 - H(f)$?

$\mathbb{P}(X=0) = \mathbb{P}(X=1) = \frac{1}{2}$ and $\mathbb{P}(Y \neq X \mid X) = 0.1/0.5 = 0.2$ whichever bit is sent, so $f = 0.2$. $C = 1 - H(0.2) = 1 - 0.722 = 0.278$ bits per bit sent: the mutual information of part (a), because equally likely inputs achieve the capacity.

[3 marks]

(c) Let p be a fair coin and q a coin with $\mathbb{P}(\text{head}) = 0.9$. Find $\mathrm{KL}(p\|q)$ and $\mathrm{KL}(q\|p)$ in bits. What does $\mathrm{KL}(p\|q)$ mean in terms of code lengths?

$\mathrm{KL}(p\|q) = \frac{1}{2}\log_2 \frac{0.5}{0.9} + \frac{1}{2}\log_2 \frac{0.5}{0.1} \approx 0.737$ bits and
$\mathrm{KL}(q\|p) = 0.9 \log_2 \frac{0.9}{0.5} + 0.1 \log_2 \frac{0.1}{0.5} \approx 0.531$ bits. They differ: the KL divergence is not symmetric (so it is not a distance). $\mathrm{KL}(p\|q)$ is the expected number of extra bits per toss when we code a fair coin with the code that is optimal for the biased one.

[4 marks]

(d) For some input, the true label distribution over three classes is $p = (0.7, 0.2, 0.1)$ and a classifier predicts $q = (0.6, 0.3, 0.1)$. Using natural logarithms, find the expected cross-entropy loss $\mathbb{E}_{Y\sim p}\big[-\ln q(Y)\big]$, the entropy H_p and $\mathrm{KL}(p\|q)$, and check that the loss is $H_p + \mathrm{KL}(p\|q)$.

Loss $= -(0.7\ln 0.6 + 0.2\ln 0.3 + 0.1\ln 0.1) \approx 0.829$ nats.
$H_p = -(0.7\ln 0.7 + 0.2\ln 0.2 + 0.1\ln 0.1) \approx 0.802$ nats.
$\mathrm{KL}(p\|q) = 0.7\ln\frac{0.7}{0.6} + 0.2\ln\frac{0.2}{0.3} \approx 0.027$ nats, and $0.802 + 0.027 = 0.829$. Only the KL term depends on the classifier, so lowering the loss moves q towards p; the loss can never go below H_p.

[4 marks]

Answers

(e) In Python, check part (d), then show numerically that the KL divergence is never negative by computing it for 10000 random pairs of distributions on five outcomes.

```
p = np.array([0.7, 0.2, 0.1])
q = np.array([0.6, 0.3, 0.1])
ce = -(p * np.log(q)).sum()
h = -(p * np.log(p)).sum()
kl = (p * np.log(p / q)).sum()
print(ce, h, kl, h + kl)
rng = np.random.default_rng(1)
P = rng.dirichlet(np.ones(5), 10000)
Q = rng.dirichlet(np.ones(5), 10000)
print((P * np.log(P / Q)).sum(1).min())
```

This prints 0.829, 0.802, 0.027 and 0.829; the smallest of the random KL divergences is 0.005, positive as it must be.

[4 marks]

Answers

End of question 2

3 Maximum entropy with discrete variables needs only ordinary Lagrange multipliers. We use natural logarithms here, so $H = -\sum_i p_i \ln p_i$.

(a) A dice has faces $i = 1,\ldots,6$ with probabilities p_i. Maximise the entropy subject only to $\sum_i p_i = 1$, using the Lagrangian $\mathcal{L} = -\sum_i p_i \ln p_i + \lambda_0\big(\sum_i p_i - 1\big)$. What distribution do you get?

$\partial\mathcal{L}/\partial p_i = -\ln p_i - 1 + \lambda_0 = 0$, so $p_i = \mathrm{e}^{\lambda_0 - 1}$ is the same for every face, and normalising gives $p_i = \frac{1}{6}$: with no information beyond normalisation, maximum entropy gives the uniform distribution.

[4 marks]

(b) Now we are told only that the average score is μ: $\sum_i i p_i = \mu$. Add a multiplier β for this constraint and show that

$$p_i = \frac{\mathrm{e}^{\beta i}}{Z(\beta)}, \qquad Z(\beta) = \sum_{j=1}^{6} \mathrm{e}^{\beta j}.$$

With $\mathcal{L} = -\sum_i p_i \ln p_i + \lambda_0(\sum_i p_i - 1) + \beta(\sum_i i p_i - \mu)$,

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\ln p_i - 1 + \lambda_0 + \beta i = 0 \quad \Rightarrow \quad p_i = \mathrm{e}^{\lambda_0 - 1}\mathrm{e}^{\beta i}.$$

Normalising fixes $\mathrm{e}^{\lambda_0 - 1} = 1/Z(\beta)$. This is the softmax of the lecture with scores $s_i = i$ and temperature $T = 1/\beta$.

[5 marks]

(c) Which value of β gives $\mu = 3.5$? Should β be positive or negative if $\mu = 4.5$? Explain.

$\beta = 0$ gives the uniform distribution, whose mean is 3.5. For $\mu = 4.5$ the high faces must be more likely, so $\mathrm{e}^{\beta i}$ must grow with i: $\beta > 0$.

[3 marks]

(d) In Python, find the β that gives $\mu = 4.5$ (use `scipy.optimize.brentq` on $\mu(\beta) - 4.5$, or bisection), and print the probabilities and the entropy in bits.

```
from scipy.optimize import brentq
i = np.arange(1, 7)
def mean(b):
    w = np.exp(b * i)
    return (w * i).sum() / w.sum()
b = brentq(lambda b: mean(b) - 4.5, -5, 5)
p = np.exp(b * i) / np.exp(b * i).sum()
print(b, p, -(p * np.log2(p)).sum())
```

$\beta \approx 0.371$ and $p \approx (0.054, 0.079, 0.114, 0.165, 0.240, 0.347)$, with entropy 2.33 bits against $\log_2 6 \approx 2.585$ for a fair dice.

[5 marks]

Answers

(e) Why is this a sensible guess for the dice, and when might it be a poor one?

Of all distributions with mean 4.5 it assumes the least: it is the most uncertain one consistent with what we know, and (from the counting argument in the lecture) by far the most likely outcome of many throws. It is a poor guess if we know more, for instance that the dice is loaded only on one face.

[3 marks]

End of question 3

Answers

4 This question is harder and optional. We move a probability mass p_i at points x_i onto a mass q_j at points y_j, where moving a unit of mass from x_i to y_j costs d_{ij}. A transport plan $\gamma_{ij} \geq 0$ has row sums $\sum_j \gamma_{ij} = p_i$ and column sums $\sum_i \gamma_{ij} = q_j$, and the optimal transport cost is the smallest $\sum_{i,j} d_{ij}\gamma_{ij}$.

(a) In one dimension, with $d_{ij} = |x_i - y_j|$ and equal masses, the optimal plan matches the points in sorted order. Find the cost of moving equal masses $\frac{1}{3}$ at $x = (0,1,3)$ to $y = (2,4,5)$. Is the optimal plan unique?

Matching in order, $W = \frac{1}{3}(|0-2| + |1-4| + |3-5|) = \frac{7}{3} \approx 2.333$. It is not unique: matching $0 \to 4$, $1 \to 2$, $3 \to 5$ also costs $\frac{1}{3}(4+1+2) = \frac{7}{3}$.

[4 marks]

(b) The Sinkhorn algorithm sets $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$, starts from $\boldsymbol{u} = \mathbf{1}$, and repeats $\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^{\mathsf{T}}\boldsymbol{u})$, $\boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K}\boldsymbol{v})$, where $\oslash$ is element-by-element division. The plan is $\gamma_{ij} = u_i K_{ij} v_j$. Show that straight after the $\boldsymbol{v}$ update the column sums of γ are exactly q_j, and straight after the $\boldsymbol{u}$ update the row sums are exactly p_i.

$\sum_i u_i K_{ij} v_j = v_j(\mathbf{K}^{\mathsf{T}}\boldsymbol{u})_j = q_j$, since $v_j = q_j/(\mathbf{K}^{\mathsf{T}}\boldsymbol{u})_j$. Likewise $\sum_j u_i K_{ij} v_j = u_i(\mathbf{K}\boldsymbol{v})_i = p_i$. Each update fixes one set of sums and slightly upsets the other; repeating converges to a plan with both.

[4 marks]

Answers

(c) Implement Sinkhorn in PyTorch for part (a) (start with `torch.set_default_dtype(torch.float64)`: $e^{-d/\epsilon}$ is tiny). Run 1000 iterations for $\epsilon = 1$, 0.1 and 0.01, and print the cost $\sum_{ij} d_{ij}\gamma_{ij}$ and the row and column sums. Look at the plan for $\epsilon = 0.01$.

```
import torch
torch.set_default_dtype(torch.float64)
x = torch.tensor([0.0, 1.0, 3.0])
y = torch.tensor([2.0, 4.0, 5.0])
p = torch.full((3,), 1/3)
q = torch.full((3,), 1/3)
d = (x[:, None] - y[None, :]).abs()
for eps in [1.0, 0.1, 0.01]:
    K = torch.exp(-d / eps)
    u = torch.ones(3)
    for _ in range(1000):
        v = q / (K.T @ u)
        u = p / (K @ v)
    gamma = u[:, None] * K * v[None, :]
    print(eps, (gamma * d).sum().item())
    print(gamma.sum(1), gamma.sum(0))
print(gamma)
```

The costs are 2.400, 2.333 and 2.333: as $\epsilon \to 0$ we recover $\frac{7}{3}$. The sums are all $\frac{1}{3}$. For $\epsilon = 0.01$ the plan is

$$\gamma \approx \begin{pmatrix} 1/6 & 1/12 & 1/12 \\ 1/6 & 1/12 & 1/12 \\ 0 & 1/6 & 1/6 \end{pmatrix},$$

not a matching. Many plans have the optimal cost here, and the entropy term picks the most spread out of them.

[7 marks]

Answers

(d) Because every step is a PyTorch operation, the Sinkhorn cost can be differentiated. Put your code in a function `sinkhorn_cost(x, y, eps=0.1, iters=200)` for points in two dimensions, with equal masses and squared distances $d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{y}_j\|^2$. Take 30 target points $\boldsymbol{y}_j$ equally spaced on the circle of radius 1 about $(3,0)$, start 30 points $\boldsymbol{x}_i$ at random (`torch.randn(30, 2, requires_grad=True)`), and run 300 steps of `torch.optim.SGD([x], lr=0.5)` on the Sinkhorn cost. Where do the points end up? Why squared distances?

```
def sinkhorn_cost(x, y, eps=0.1, iters=200):
    d = ((x[:, None, :] - y[None, :, :])**2).sum(-1)
    K = torch.exp(-d / eps)
    p = torch.full((len(x),), 1/len(x))
    q = torch.full((len(y),), 1/len(y))
    u = torch.ones(len(x))
    for _ in range(iters):
        v = q / (K.T @ u)
        u = p / (K @ v)
    return (u[:, None] * K * v[None, :] * d).sum()
torch.manual_seed(0)
t = torch.linspace(0, 2*torch.pi, 31)[:-1]
y = torch.stack([3 + torch.cos(t), torch.sin(t)], 1)
x = torch.randn(30, 2, requires_grad=True)
opt = torch.optim.SGD([x], lr=0.5)
for step in range(301):
    opt.zero_grad()
    loss = sinkhorn_cost(x, y)
    loss.backward()
    opt.step()
    if step % 50 == 0:
        print(step, loss.item())
```

The cost falls from 10.7 to 0.051, and the points end up on the circle (mean distance from $(3,0)$ 1.000, spread 0.0004), one near each target. The small cost that remains comes from the entropy term. The distance $\|\boldsymbol{x} - \boldsymbol{y}\|$ has no gradient when a point lands exactly on a target, while its square is smooth everywhere. This is the idea behind training generative models with optimal transport: the points are the model's samples, and the gradient moves them towards the data.

[10 marks]

End of question 4

END OF PAPER